Лабораторная работа 5. Unit-тестирование

Имя студента: Цароев Альберт Казебкович
Группа: ПИЖ-б-о-23-1(1)
Предмет: Тестирование и отладка программного обеспечения
Претендую на оценку: Отлично

---

1. Описание метода и стратегии тестирования

1.1. Метод

Unit-тест (модульный тест) проверяет одну наименьшую единицу кода в изоляции от остальных.
Свойства хорошего unit-теста (FIRST):

Fast	миллисекунды: запускается после каждого изменения
Independent	не зависит от других тестов и порядка их запуска
Repeatable	одинаковый результат при каждом запуске: без случайности, реального времени, сети
Self-validating	результат - прошёл или упал, без ручного просмотра вывода
Timely	пишется вместе с кодом

Структура каждого теста - **Arrange** / **Act** / **Assert**: подготовить данные, выполнить одно действие,
проверить результат.

Зависимости единицы заменяются **тестовыми двойниками**:

Двойник	Назначение	В проекте
stub (заглушка)	возвращает заранее заданный ответ	`SpySpawner.answer`
spy (шпион)	запоминает, как его вызвали	`SpySpawner.seen` - какие свободные клетки ему передали
fake (подделка)	упрощённая рабочая реализация	`ScriptedSpawner` (labs/common) - еда по списку

Подменить случайный выбор еды можно потому, что он скрыт за трейтом `FoodSpawner`, а ресурс `FoodSource`
хранит `Box<dyn FoodSpawner>`.

1.2. Unit-тесты в Rust и Bevy

Rust. Модульные тесты - это модуль под `#[cfg(test)]` *внутри* крейта. Такой модуль видит закрытые элементы
родителя и компилируется только при `cargo test`. Обычно его пишут в том же файле (`mod tests { … }`).
Здесь файлы тестов лежат в `labs/lb5/`, рядом с отчётом, а в код подключаются атрибутом `#[path]`:

```
// src/game/mod.rs
#[cfg(test)]
#[path = "../../labs/lb5/system_units.rs"]
mod lb5;
```

Для компилятора это обычный дочерний модуль `crate::game::lb5`. Поэтому тесты видят системы `logic::*`,
закрытые для внешнего кода (`pub(super)`). Тестам из `labs/lb1…lb4, lb6` они недоступны.

Bevy. Единица в ECS - **одна система**. Тест не запускает приложение с расписанием и плагинами,
а создаёт пустой `World`, кладёт в него минимум ресурсов и сущностей и вызывает систему

один раз:

```
let mut world = world_with_snake(Board::new(5, 5), &start, Direction::Right);   // Arrange
world.run_system_once(move_snake).unwrap();                                      // Act
assert_eq!(snake_cells(&world), vec![Cell::new(3, 2), Cell::new(2, 2), Cell::new(1, 2)]);
```

Сообщения, которые написала система, читаются прямо из ресурса `Messages<T>`.

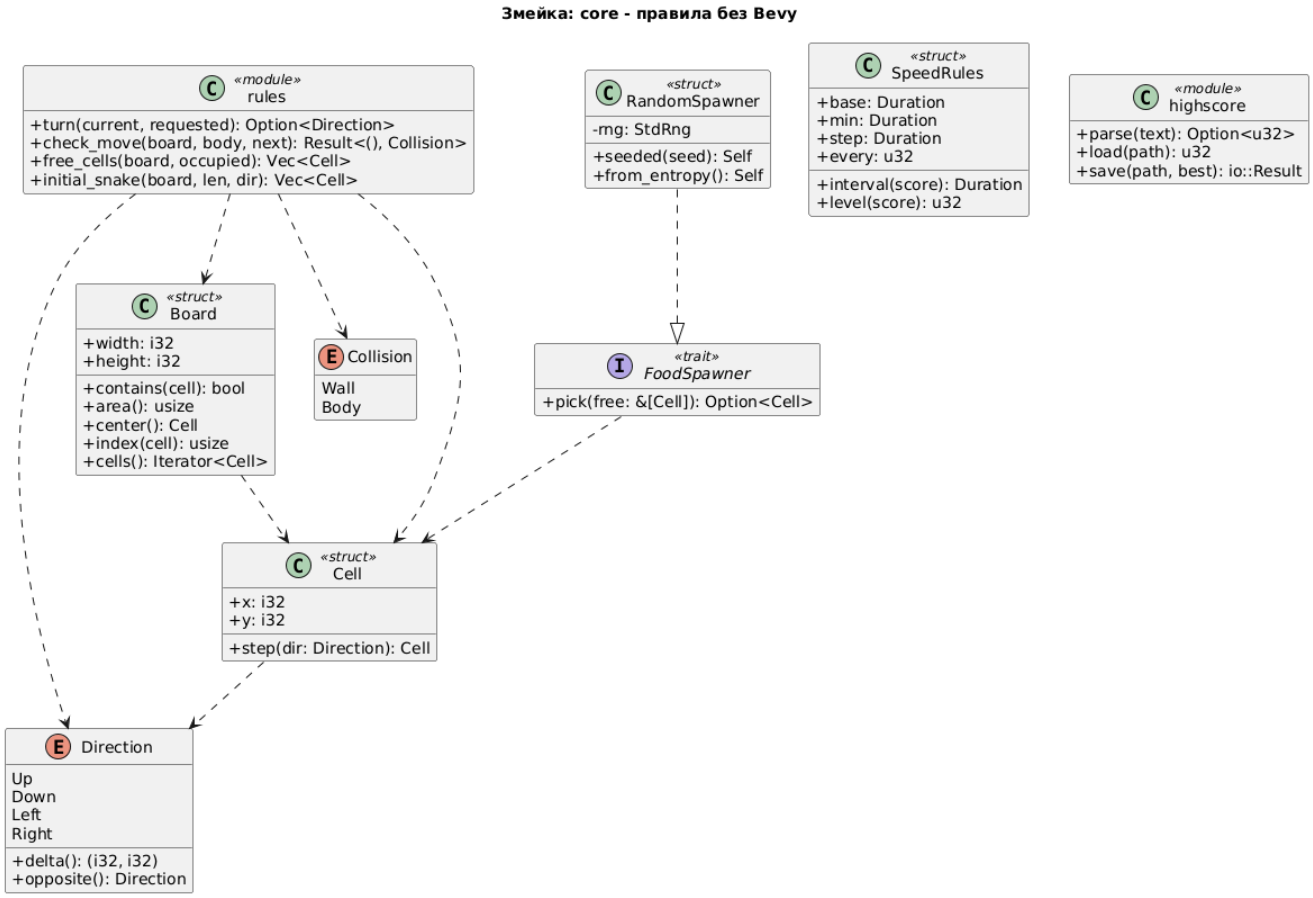
1.3. Стратегия

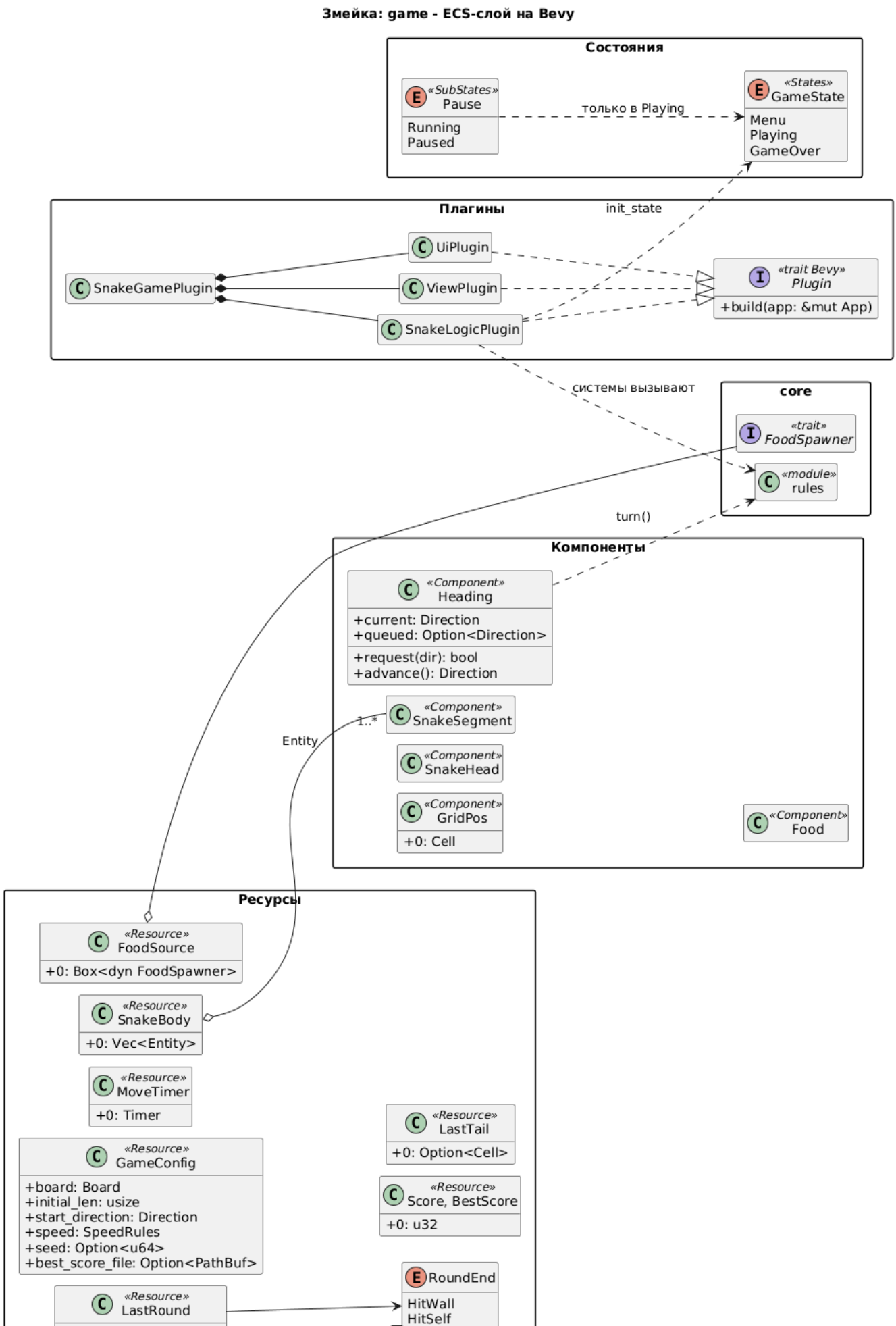
Уровень	Единица	Файл
ядро	функции и типы `core`: `Cell`, `Direction`, `Board`, `turn`, `initial_snake`, `free_cells`, `RandomSpawner`	core_units.rs
компонент	методы `Heading::request` / `advance`	system_units.rs
система	`move_snake`, `eat_food`, `spawn_food` по отдельности в пустом `World`	system_units.rs

1.4. Запуск

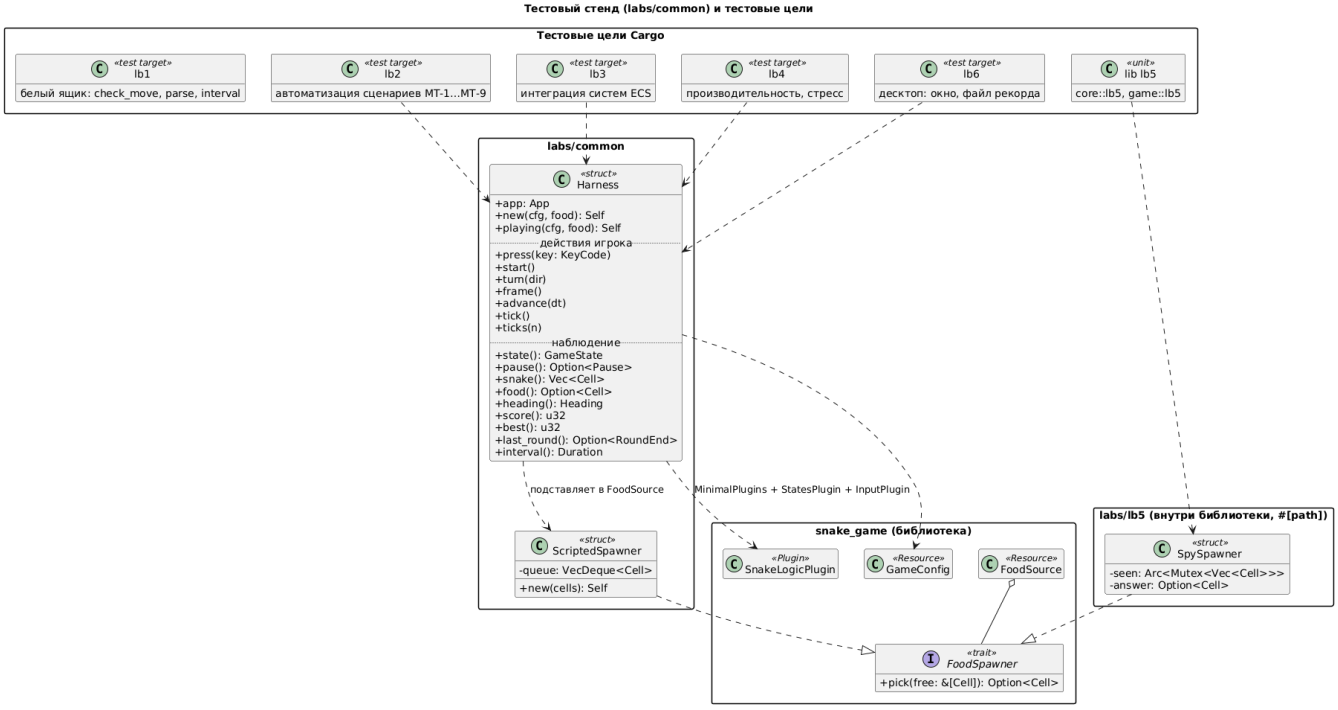
```
cargo test --lib lb5                    # все unit-тесты работы (core::lb5::*, game::lb5
cargo test --lib game::lb5              # только системы
scripts/test.sh lb5 --report
```

2. Диаграмма классов




Змейка: game - ECS-слой на Bevy
Состояния
«SubStates»
Pause
Running
Paused
только в Playing
«States»
GameState
Menu
Playing
GameOver
Плагины
init_state
UiPlugin
SnakeGamePlugin
ViewPlugin
SnakeLogicPlugin
«trait Bevy»
Plugin
+build(app: &mut App)
системы вызывают
core
«trait»
FoodSpawner
«module»
rules
turn()
Компоненты
«Component»
Heading
+current: Direction
+queued: Option<Direction>
+request(dir): bool
+advance(): Direction
1..*
Entity
«Component»
SnakeSegment
«Component»
SnakeHead
«Component»
GridPos
+0: Cell
«Component»
Food
Ресурсы
«Resource»
FoodSource
+0: Box<dyn FoodSpawner>
«Resource»
SnakeBody
+0: Vec<Entity>
«Resource»
MoveTimer
+0: Timer
«Resource»
LastTail
+0: Option<Cell>
«Resource»
GameConfig
+board: Board
+initial_len: usize
+start_direction: Direction
+speed: SpeedRules
+seed: Option<u64>
+best_score_file: Option<PathBuf>
«Resource»
Score, BestScore
+0: u32
RoundEnd
HitWall
HitSelf
«Resource»
LastRound

Тестовые двойники `SpySpawner` и `ScriptedSpawner` реализуют тот же трейт `FoodSpawner`, что и `RandomSpawner`:



3. Тестовые материалы

Тест	Единица	Пр
`step_moves_by_one_cell`	`Cell::step`	4 направл соседние
`opposite_is_involution_and_steps_back`	`Direction::opposite`	дважды противопо исходное; обратно =
`board_cells_cover_area_in_index_order`	`Board::cells`, `index`	число клет индексы 0 порядку
`degenerate_board_has_no_cells`	`Board::area`	поле 0×5 и клеток

Тест	Единица	Пр
`turn_allows_only_perpendicular`	`rules::turn`	вправо → вправо/вле
`initial_snake_trails_behind_head`	`rules::initial_snake`	20×15, дли (9,7),(8,7),
`initial_snake_is_clipped_by_board`	`rules::initial_snake`	4×1, длина клетки
`free_cells_excludes_snake_and_ignores_outside`	`rules::free_cells`	клетка вне «занятых» расчёт
`random_spawner_picks_only_from_free`	`RandomSpawner`	100 выбор списка сво
`random_spawner_none_when_board_full`	`RandomSpawner`	пустой спи
`random_spawner_is_reproducible_by_seed`	`RandomSpawner`	одинаковы одинакова последова
`heading_rejects_reverse_even_after_queued_turn`	`Heading::request`	вправо, ↑ отклонён (относител
`heading_advance_consumes_queue_once`	`Heading::advance`	поворот пр один раз
`move_snake_shifts_every_segment`	система `move_snake`	все сегмен `LastTail`
`move_snake_reports_wall_and_stays`	система `move_snake`	`RoundOve` змейка не
`eat_food_scores_and_despawns`	система `eat_food`	счёт +1, ед `FoodEate`
`eat_food_ignores_food_elsewhere`	система `eat_food`	еда в друг

Тест	Единица	Пр
		съедается
`spawn_food_offers_only_free_cells_and_places_answer`	система `spawn_food` + spy	шпиону пе свободны клетке отв
`spawn_food_without_place_wins_round`	система `spawn_food` + stub	заглушка в → `RoundOve`

4. Результаты выполнения тестов

Все 19 тестов пройдены. Протокол: results.txt.

```
Протокол: lb5
Система:  Linux 7.0.0-38-generic x86_64
Rust:     rustc 1.98.1 (48a229cea 2026-09-01)
Команда:  cargo test --lib lb5

test core::lb5::board_cells_cover_area_in_index_order ... ok
test core::lb5::degenerate_board_has_no_cells ... ok
test core::lb5::free_cells_excludes_snake_and_ignores_outside ... ok
test core::lb5::initial_snake_is_clipped_by_board ... ok
test core::lb5::initial_snake_trails_behind_head ... ok
test core::lb5::opposite_is_involution_and_steps_back ... ok
test core::lb5::random_spawner_is_reproducible_by_seed ... ok
test core::lb5::random_spawner_none_when_board_full ... ok
test core::lb5::random_spawner_picks_only_from_free ... ok
test core::lb5::step_moves_by_one_cell ... ok
test core::lb5::turn_allows_only_perpendicular ... ok
test game::lb5::eat_food_ignores_food_elsewhere ... ok
test game::lb5::eat_food_scores_and_despawns ... ok
test game::lb5::heading_advance_consumes_queue_once ... ok
test game::lb5::heading_rejects_reverse_even_after_queued_turn ... ok
test game::lb5::move_snake_reports_wall_and_stays ... ok
test game::lb5::move_snake_shifts_every_segment ... ok
test game::lb5::spawn_food_offers_only_free_cells_and_places_answer ... ok
test game::lb5::spawn_food_without_place_wins_round ... ok

test result: ok. 19 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Проверка тестов мутацией

Мутация	Результат
`Heading::request` сверяет поворот с `queued`, а не с `current` (разворот двумя быстрыми нажатиями)	упал `heading_rejects_reverse_even_after_queued_turn`, остальные 18 прошли